🔥 PySpark Complete Cheat Sheet

From Setup to Production — with Explanations for Every Concept

> **What is PySpark?** PySpark is the Python API for Apache Spark — a distributed computing framework that lets you process massive datasets across a cluster of machines. It uses a concept called *lazy evaluation*: transformations are not executed until an *action* is called. This makes it highly efficient for big data workloads.

📋 Table of Contents

1. Installation & Setup

PySpark can be installed via pip. You also need Java (JDK 8 or 11) installed on your machine since Spark runs on the JVM. In production, Spark runs on a cluster (YARN, Kubernetes, Mesos), but locally you can run it in *local mode* for development and testing.

Install PySpark

```
pip install pyspark
pip install pyspark[sql]       # includes SQL extras
pip install pyspark[streaming] # includes streaming
pip install delta-spark        # for Delta Lake support
```

Environment Variables

```
import os
os.environ["JAVA_HOME"] = "/usr/lib/jvm/java-11-openjdk"
os.environ["SPARK_HOME"] = "/opt/spark"
os.environ["PYSPARK_PYTHON"] = "python3"
os.environ["PYSPARK_DRIVER_PYTHON"] = "python3"
```

💡 **Tip:** Use `findspark` library to auto-locate Spark: `import findspark; findspark.init()`

2. SparkSession & SparkContext

SparkSession is the unified entry point for all Spark functionality (SQL, DataFrames, Datasets). It was introduced in Spark 2.0 to replace the older `SparkContext`, `SQLContext`, and `HiveContext`. Think of it as your "connection" to the Spark engine.

SparkContext is the lower-level entry point for RDD operations. It's still accessible via `spark.sparkContext`.

Create SparkSession

```
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("MyApp") \
    .master("local[*]") \       # local[*] = use all CPU cores
    .config("spark.executor.memory", "4g") \
    .config("spark.driver.memory", "2g") \
    .config("spark.sql.shuffle.partitions", "200") \
    .getOrCreate()

sc = spark.sparkContext          # access SparkContext
sc.setLogLevel("WARN")           # reduce log noise
```

Master URL Options

Master URL	Description
`local`	Single thread (no parallelism)
`local[N]`	N threads locally
`local[*]`	All available CPU cores
`yarn`	YARN cluster (Hadoop)
`spark://host:7077`	Standalone Spark cluster
`k8s://https://host`	Kubernetes cluster

Stop SparkSession

```
spark.stop()   # always stop when done to free resources
```

3. Reading Data

> Spark can read data from many sources: local files, HDFS, S3, databases, Kafka, etc. The `spark.read` API returns a **DataFrame** — a distributed table with named columns. Reading is lazy: Spark only scans the data when an action is triggered. Always specify a schema when possible to avoid the expensive *schema inference* scan.

Read CSV

```
df = spark.read \
    .option("header", "true") \       # first row = column names
    .option("inferSchema", "true") \  # auto-detect types (slow on large files)
    .option("sep", ",") \             # delimiter
    .option("nullValue", "NA") \      # treat "NA" as null
    .option("multiLine", "true") \    # handle multi-line fields
    .csv("path/to/file.csv")

# Read multiple files
df = spark.read.csv(["file1.csv", "file2.csv"], header=True)
df = spark.read.csv("path/to/folder/*.csv", header=True)
```

Read JSON

```
df = spark.read.json("path/to/file.json")
df = spark.read \
    .option("multiLine", "true") \    # for pretty-printed JSON
    .json("path/to/file.json")
```

Read Parquet (Recommended Format)

```
# Parquet is columnar, compressed, and schema-preserving — best for Spark
df = spark.read.parquet("path/to/file.parquet")
df = spark.read.parquet("s3://bucket/path/")  # from S3
```

Read ORC

```
df = spark.read.orc("path/to/file.orc")
```

Read from JDBC (Database)

```
df = spark.read \
    .format("jdbc") \
    .option("url", "jdbc:postgresql://host:5432/dbname") \
    .option("dbtable", "schema.tablename") \
    .option("user", "username") \
    .option("password", "password") \
    .option("driver", "org.postgresql.Driver") \
    .load()
```

Read from Hive

```
df = spark.sql("SELECT * FROM hive_db.table_name")
spark.catalog.listTables()    # list available tables
```

Create DataFrame from Python Data

```
from pyspark.sql import Row

# From list of tuples
df = spark.createDataFrame([(1, "Alice", 30), (2, "Bob", 25)],
                           ["id", "name", "age"])

# From list of dicts
data = [{"id": 1, "name": "Alice"}, {"id": 2, "name": "Bob"}]
df = spark.createDataFrame(data)

# From Pandas DataFrame
import pandas as pd
pdf = pd.DataFrame({"a": [1, 2], "b": [3, 4]})
df = spark.createDataFrame(pdf)
```

4. Writing Data

> Writing data in Spark is an **action** — it triggers execution of all pending transformations. The `mode` parameter controls what happens if the destination already exists. Always choose the right format: Parquet for analytics, CSV/JSON for interoperability, JDBC for databases.

Write Modes

Mode	Behavior
`overwrite`	Delete existing data and write new
`append`	Add to existing data
`ignore`	Do nothing if data exists
`error` (default)	Throw error if data exists

Write CSV / JSON / Parquet / ORC

```
df.write.mode("overwrite").csv("output/path", header=True)
df.write.mode("append").json("output/path")
df.write.mode("overwrite").parquet("output/path")
df.write.mode("overwrite").orc("output/path")
```

Write with Partitioning (Partition by Column)

```
# Saves data into subdirectories: output/year=2024/month=01/...
df.write.partitionBy("year", "month") \
    .mode("overwrite") \
    .parquet("output/path")
```

> **Partition by column** when writing splits the output files into subdirectories based on column values. This is called *Hive-style partitioning*. It dramatically speeds up future reads when you filter on those columns — Spark only reads the relevant subdirectories (partition pruning).

Write to JDBC

```
df.write \
    .format("jdbc") \
    .option("url", "jdbc:postgresql://host:5432/db") \
    .option("dbtable", "target_table") \
    .option("user", "user") \
    .option("password", "pass") \
    .mode("append") \
    .save()
```

Control Output File Count

```
df.coalesce(1).write.csv("output/")    # write to single file
df.repartition(10).write.parquet("output/")   # write to 10 files
```

5. Schema & Data Types

> A **schema** defines the structure of a DataFrame: column names and their data types. Defining schemas explicitly is a best practice — it avoids the slow schema inference scan and prevents type mismatches. Spark uses its own type system (StructType, StringType, etc.) that maps to SQL types.

Define Schema Manually

```
from pyspark.sql.types import (
    StructType, StructField, StringType, IntegerType,
    DoubleType, BooleanType, LongType, TimestampType,
    DateType, ArrayType, MapType, FloatType
)

schema = StructType([
    StructField("id",        LongType(),      nullable=False),
    StructField("name",      StringType(),    nullable=True),
    StructField("age",       IntegerType(),   nullable=True),
    StructField("salary",    DoubleType(),    nullable=True),
    StructField("is_active", BooleanType(),   nullable=True),
    StructField("joined",    TimestampType(), nullable=True),
    StructField("tags",      ArrayType(StringType()), nullable=True),
    StructField("meta",      MapType(StringType(), StringType()), nullable=True),
])

df = spark.read.schema(schema).csv("data.csv", header=True)
```

Schema from DDL String

```
schema = "id LONG, name STRING, age INT, salary DOUBLE"
df = spark.read.schema(schema).csv("data.csv", header=True)
```

Inspect Schema

```
df.printSchema()        # tree view of schema
df.schema               # StructType object
df.dtypes               # list of (column_name, type_string) tuples
df.columns              # list of column names
```

Cast / Change Column Type

```
from pyspark.sql.functions import col

df = df.withColumn("age", col("age").cast(IntegerType()))
df = df.withColumn("salary", col("salary").cast("double"))
df = df.withColumn("joined", col("joined").cast("timestamp"))
```

Common Data Types

PySpark Type	Python Equivalent	SQL Type

StringType()	str	STRING / VARCHAR
IntegerType()	int (32-bit)	INT
LongType()	int (64-bit)	BIGINT
FloatType()	float (32-bit)	FLOAT
DoubleType()	float (64-bit)	DOUBLE
BooleanType()	bool	BOOLEAN
DateType()	datetime.date	DATE
TimestampType()	datetime.datetime	TIMESTAMP
ArrayType(T)	list	ARRAY<T>
MapType(K,V)	dict	MAP<K,V>
StructType()	dict/object	STRUCT

6. DataFrame Basics & Inspection

> A **DataFrame** is Spark's primary abstraction — a distributed, immutable table of rows and columns. It is similar to a Pandas DataFrame but lives across a cluster. All transformations return a new DataFrame (immutability). Use inspection methods to understand your data before processing.

Basic Inspection

```
df.show()                   # print first 20 rows
df.show(50, truncate=False)  # show 50 rows, no truncation
df.show(10, vertical=True)   # vertical display (one row per block)
df.head(5)                   # return first 5 rows as list of Row objects
df.first()                  # return first row
df.take(10)                  # return first 10 rows as list
df.collect()                 # return ALL rows — use carefully on large data!
df.count()                   # count total rows (triggers execution)
df.describe().show()         # summary stats: count, mean, stddev, min, max
df.summary().show()          # extended stats including percentiles
```

DataFrame Metadata

```
df.columns          # ['col1', 'col2', ...]
df.dtypes           # [('col1', 'string'), ('col2', 'int'), ...]
df.schema           # StructType(...)
df.printSchema()    # pretty-print schema tree
len(df.columns)     # number of columns
df.rdd.getNumPartitions()  # number of partitions
```

Add / Rename / Drop Columns

```
df = df.withColumn("new_col", col("salary") * 1.1)   # add/replace column
df = df.withColumnRenamed("old_name", "new_name")     # rename column
df = df.drop("col1", "col2")                          # drop columns
df = df.toDF("a", "b", "c")                           # rename all columns
```

7. Selecting & Filtering

> **select()** picks specific columns (like SQL SELECT). **filter()** / **where()** keeps only rows matching a condition (like SQL WHERE). These are *transformations* — they are lazy and don't execute until an action is called.

Select Columns

```
from pyspark.sql.functions import col, expr

df.select("name", "age")                        # by name
df.select(col("name"), col("age") + 1)          # with expressions
df.select("*")                                  # all columns
```

```
df.select([c for c in df.columns if c != "id"]) # all except 'id'
df.selectExpr("name", "age + 1 as age_plus_one", "salary * 12 as annual")
```

Filter / Where

```
df.filter(col("age") > 25)
df.filter("age > 25")                          # SQL string syntax
df.where(col("name") == "Alice")
df.filter((col("age") > 25) & (col("salary") > 50000))  # AND
df.filter((col("age") < 20) | (col("age") > 60))         # OR
df.filter(~col("is_active"))                             # NOT
df.filter(col("name").isin("Alice", "Bob", "Charlie"))    # IN list
df.filter(col("name").like("%ali%"))                      # LIKE pattern
df.filter(col("name").rlike("^A.*"))                      # regex
df.filter(col("age").between(20, 40))                     # BETWEEN
```

8. Column Operations & Expressions

> Spark provides a rich set of built-in functions via `pyspark.sql.functions`. These are optimized native functions that run on the JVM — always prefer them over Python UDFs for performance. The `col()` function references a column by name, and `expr()` lets you write SQL-style expressions as strings.

Import Functions

```
from pyspark.sql import functions as F
# or import individually:
from pyspark.sql.functions import col, lit, expr, when, coalesce
```

Conditional Logic (CASE WHEN)

```
# when().otherwise() is equivalent to SQL CASE WHEN
df = df.withColumn("category",
    F.when(col("age") < 18, "minor")
     .when(col("age") < 65, "adult")
     .otherwise("senior")
)

# Using expr (SQL string)
df = df.withColumn("grade",
    F.expr("CASE WHEN score >= 90 THEN 'A' WHEN score >= 80 THEN 'B' ELSE 'C' END")
)
```

Arithmetic & Math

```
df.withColumn("total", col("price") * col("qty"))
df.withColumn("log_val", F.log(col("value")))
df.withColumn("sqrt_val", F.sqrt(col("value")))
df.withColumn("abs_val", F.abs(col("value")))
df.withColumn("rounded", F.round(col("price"), 2))
df.withColumn("floored", F.floor(col("price")))
df.withColumn("ceiled", F.ceil(col("price")))
df.withColumn("power", F.pow(col("base"), 2))
```

Literal Values

```
df.withColumn("constant", F.lit(42))
df.withColumn("flag", F.lit(True))
df.withColumn("label", F.lit("processed"))
```

9. Aggregations & GroupBy

> **GroupBy** splits the DataFrame into groups based on column values, then applies an aggregation function to each group — just like SQL GROUP BY. Aggregations are *wide transformations* that require a **shuffle**

(data movement across the network), which is expensive. The number of shuffle partitions is controlled by `spark.sql.shuffle.partitions` (default: 200).

GroupBy + Aggregate

```
from pyspark.sql.functions import count, sum, avg, min, max, countDistinct,
collect_list

df.groupBy("department").count()
df.groupBy("department").sum("salary")
df.groupBy("department").avg("salary")
df.groupBy("department", "year").agg(
    count("*").alias("total_employees"),
    F.sum("salary").alias("total_salary"),
    F.avg("salary").alias("avg_salary"),
    F.max("salary").alias("max_salary"),
    F.min("salary").alias("min_salary"),
    F.countDistinct("role").alias("unique_roles"),
    F.collect_list("name").alias("names"),
    F.collect_set("role").alias("unique_role_set"),
    F.stddev("salary").alias("salary_stddev"),
    F.variance("salary").alias("salary_variance"),
    F.first("name").alias("first_name"),
    F.last("name").alias("last_name"),
    F.percentile_approx("salary", 0.5).alias("median_salary")
)
```

Pivot Table

```
# Pivot: turn row values into columns
df.groupBy("department") \
  .pivot("year", [2022, 2023, 2024]) \
  .agg(F.sum("salary")) \
  .show()
```

Rollup & Cube (Multi-level Aggregation)

```
# Rollup: hierarchical subtotals
df.rollup("country", "city").agg(F.sum("sales")).show()

# Cube: all combinations of subtotals
df.cube("country", "city").agg(F.sum("sales")).show()
```

10. Joins

> A **join** combines two DataFrames based on a matching condition. Joins are one of the most expensive operations in Spark because they require a **shuffle** — data with the same key must be on the same partition. The most important optimization is the **broadcast join**: if one DataFrame is small enough to fit in memory, Spark sends a copy to every executor, completely avoiding the shuffle.

Join Types

Join Type	Description
`inner`	Only matching rows from both sides
`left` / `left_outer`	All rows from left, matched from right (null if no match)
`right` / `right_outer`	All rows from right, matched from left
`full` / `full_outer`	All rows from both sides
`left_semi`	Rows from left where match exists in right (no right columns)
`left_anti`	Rows from left where NO match exists in right
`cross`	Cartesian product (every row × every row)

Join Syntax

```
joined = df1.join(df2, on="id", how="inner")
joined = df1.join(df2, on=["id", "dept"], how="left")
joined = df1.join(df2, df1.id == df2.emp_id, how="inner")

# Avoid ambiguous columns after join
joined = df1.join(df2, on="id").drop(df2.id)
```

Broadcast Join (Small Table Optimization)

```
from pyspark.sql.functions import broadcast

# Broadcast the small lookup table to all executors
result = large_df.join(broadcast(small_df), on="id", how="left")

# Set auto-broadcast threshold (default 10MB)
spark.conf.set("spark.sql.autoBroadcastJoinThreshold", 50 * 1024 * 1024)  # 50MB
```

> 💡 **Tip:** Always broadcast the smaller DataFrame in a join. This eliminates the shuffle and can speed up joins by 10-100x.

11. Sorting & Ordering

> **orderBy()** / **sort()** sorts the entire DataFrame globally — this requires collecting all data to a single partition, which is expensive. Use it only when you truly need a global order. For local ordering within groups, use **Window functions** instead.

```
df.orderBy("age")                                # ascending (default)
df.orderBy(col("age").desc())                    # descending
df.orderBy(col("dept").asc(), col("salary").desc())  # multi-column
df.sort("age", ascending=False)                  # alternative syntax
df.orderBy(col("name").asc_nulls_last())         # nulls at end
df.orderBy(col("name").desc_nulls_first())      # nulls at start
```

12. Null Handling

> In distributed systems, missing data is common. Spark represents missing values as `null`. Proper null handling is critical — many functions return null if any input is null. Use `coalesce()` to provide fallback values, `dropna()` to remove nulls, and `fillna()` to replace them.

Check for Nulls

```
df.filter(col("name").isNull())
df.filter(col("name").isNotNull())
df.select([F.count(F.when(F.col(c).isNull(), c)).alias(c) for c in df.columns]).show()
```

Drop Rows with Nulls

```
df.dropna()                          # drop rows with ANY null
df.dropna(how="all")                 # drop rows where ALL values are null
df.dropna(subset=["name", "age"])    # drop rows with null in specific columns
df.dropna(thresh=3)                  # keep rows with at least 3 non-null values
```

Fill Nulls

```
df.fillna(0)                         # fill all numeric nulls with 0
df.fillna("unknown")                 # fill all string nulls
df.fillna({"age": 0, "name": "N/A", "salary": df.agg(F.avg("salary")).first()[0]})
```

Coalesce (First Non-Null)

```
# Returns the first non-null value from the list of columns
df.withColumn("display_name",
    F.coalesce(col("preferred_name"), col("full_name"), F.lit("Anonymous"))
)
```

Replace Values

```
df.replace("N/A", None)              # replace string with null
df.replace({"old_val": "new_val"})   # replace in all columns
df.na.replace(0, -1, subset=["age"]) # replace in specific column
```

13. String Functions

> PySpark provides a comprehensive set of string manipulation functions. These are vectorized operations that run efficiently across the cluster — much faster than applying Python string methods row by row.

```
F.upper(col("name"))                 # UPPERCASE
F.lower(col("name"))                 # lowercase
F.trim(col("name"))                  # remove leading/trailing spaces
F.ltrim(col("name"))                 # remove leading spaces
F.rtrim(col("name"))                 # remove trailing spaces
```

```
F.length(col("name"))                    # string length
F.substring(col("name"), 1, 3)           # substring(col, start, length) — 1-indexed
F.concat(col("first"), F.lit(" "), col("last"))  # concatenate
F.concat_ws("-", col("a"), col("b"))     # concat with separator
F.split(col("tags"), ",")                # split string into array
F.regexp_replace(col("text"), r"\d+", "NUM")  # regex replace
F.regexp_extract(col("text"), r"(\d+)", 1)    # regex extract group 1
F.instr(col("text"), "spark")            # find position of substring
F.lpad(col("id"), 5, "0")               # left-pad: "42" -> "00042"
F.rpad(col("id"), 5, "0")               # right-pad
F.reverse(col("name"))                   # reverse string
F.translate(col("text"), "aeiou", "*****")  # char-by-char replace
F.initcap(col("name"))                   # Title Case
F.format_string("Hello %s, age %d", col("name"), col("age"))  # printf-style
```

14. Date & Time Functions

> Date and time operations are critical for time-series analysis, reporting, and ETL pipelines. Spark stores dates as `DateType` (no time) and timestamps as `TimestampType` (with time). Always be aware of timezone handling — Spark uses the JVM's default timezone unless configured otherwise.

```
F.current_date()                          # today's date
F.current_timestamp()                     # current datetime
F.to_date(col("date_str"), "yyyy-MM-dd")  # parse string to date
F.to_timestamp(col("ts_str"), "yyyy-MM-dd HH:mm:ss")  # parse to timestamp
F.date_format(col("date"), "MM/dd/yyyy")  # format date to string
F.year(col("date"))                       # extract year
F.month(col("date"))                      # extract month (1-12)
F.dayofmonth(col("date"))                 # day of month (1-31)
F.dayofweek(col("date"))                  # day of week (1=Sunday)
F.dayofyear(col("date"))                  # day of year (1-366)
F.hour(col("ts"))                         # extract hour
F.minute(col("ts"))                       # extract minute
F.second(col("ts"))                       # extract second
F.quarter(col("date"))                    # quarter (1-4)
F.weekofyear(col("date"))                 # ISO week number
F.datediff(col("end_date"), col("start_date"))  # days between dates
F.months_between(col("end"), col("start"))      # months between
F.date_add(col("date"), 7)                # add 7 days
F.date_sub(col("date"), 30)               # subtract 30 days
F.add_months(col("date"), 3)              # add 3 months
F.last_day(col("date"))                   # last day of month
F.next_day(col("date"), "Monday")         # next Monday
F.trunc(col("date"), "month")             # truncate to month start
F.date_trunc("hour", col("ts"))           # truncate timestamp to hour
F.unix_timestamp(col("ts"))               # convert to Unix epoch seconds
F.from_unixtime(col("epoch"), "yyyy-MM-dd")  # epoch to string
```

15. Array & Map Functions

> Spark supports complex nested types: **Arrays** (ordered lists) and **Maps** (key-value pairs). These are useful for storing multi-valued attributes in a single column. The `explode()` function is key — it converts an array/map into multiple rows, one per element.

```
F.array(col("a"), col("b"), col("c"))   # create array from columns
F.array_contains(col("tags"), "spark")  # check if value in array
F.array_distinct(col("tags"))           # remove duplicates
F.array_union(col("a"), col("b"))       # union of two arrays
F.array_intersect(col("a"), col("b"))   # intersection
F.array_except(col("a"), col("b"))      # elements in a but not b
F.array_sort(col("tags"))               # sort array
F.array_min(col("scores"))              # min value in array
F.array_max(col("scores"))              # max value in array
F.size(col("tags"))                     # length of array
F.slice(col("arr"), 1, 3)               # slice(arr, start, length)
```

```
F.flatten(col("nested_arr"))            # flatten nested arrays
F.explode(col("tags"))                   # one row per array element
F.explode_outer(col("tags"))             # like explode but keeps nulls
F.posexplode(col("tags"))                # explode with position index
F.collect_list(col("name"))              # aggregate rows into array
F.collect_set(col("name"))               # aggregate into unique array

# Map functions
F.create_map(col("key"), col("val"))     # create map from columns
F.map_keys(col("meta"))                  # get all keys
F.map_values(col("meta"))                # get all values
col("meta")["key_name"]                  # access map value by key
F.explode(col("meta"))                   # explode map into key/value rows
```

16. Window Functions

> **Window functions** perform calculations across a set of rows that are related to the current row — without collapsing them into a single result like GroupBy does. Think of it as "aggregation with context." Common uses: running totals, rankings, moving averages, lag/lead comparisons. A **Window spec** defines: which rows to include (PARTITION BY), in what order (ORDER BY), and the frame (ROWS/RANGE BETWEEN).

Define Window Spec

```
from pyspark.sql.window import Window

# Partition by department, order by salary descending
w = Window.partitionBy("department").orderBy(col("salary").desc())

# Unbounded window (all rows in partition)
w_all = Window.partitionBy("department") \
              .orderBy("date") \
              .rowsBetween(Window.unboundedPreceding, Window.unboundedFollowing)

# Rolling window: current row and 2 preceding rows
w_rolling = Window.partitionBy("id") \
                  .orderBy("date") \
                  .rowsBetween(-2, 0)
```

Ranking Functions

```
df.withColumn("rank",       F.rank().over(w))        # gaps for ties: 1,2,2,4
df.withColumn("dense_rank", F.dense_rank().over(w))  # no gaps: 1,2,2,3
df.withColumn("row_number", F.row_number().over(w))  # unique: 1,2,3,4
df.withColumn("ntile",      F.ntile(4).over(w))      # quartile bucket
df.withColumn("percent_rank", F.percent_rank().over(w))  # 0.0 to 1.0
```

Analytic Functions

```
df.withColumn("running_total", F.sum("sales").over(w))
df.withColumn("running_avg",   F.avg("sales").over(w))
df.withColumn("cumulative_max", F.max("sales").over(w))
df.withColumn("lag_1",  F.lag("sales", 1, 0).over(w))   # previous row value
df.withColumn("lead_1", F.lead("sales", 1, 0).over(w))  # next row value
df.withColumn("first_val", F.first("sales").over(w_all))
df.withColumn("last_val",  F.last("sales").over(w_all))
```

17. Partitioning & Repartitioning

Partitioning is how Spark divides data across the cluster. Each partition is a chunk of data processed by one task on one executor. The number of partitions directly affects parallelism and performance.

repartition(n) — Performs a full shuffle to redistribute data into exactly *n* partitions. Use when you need to increase partitions or evenly distribute data by a specific column. It's expensive (full network shuffle) but produces balanced partitions.

coalesce(n) — Reduces the number of partitions by merging existing ones, *without a full shuffle*. Much cheaper than repartition, but can create uneven partitions. Use when writing output to fewer files.

Rule of thumb: Aim for partition sizes of 128MB–256MB. Too few partitions = underutilized cluster. Too many = overhead from task scheduling.

Check & Change Partitions

```
df.rdd.getNumPartitions()          # check current partition count

# Increase partitions (full shuffle)
df_repartitioned = df.repartition(100)

# Repartition by column (data with same key goes to same partition)
df_repartitioned = df.repartition(50, col("department"))
df_repartitioned = df.repartition(col("country"), col("city"))

# Decrease partitions (no shuffle — just merge)
df_coalesced = df.coalesce(10)

# Optimal partition count: 2-4x number of CPU cores in cluster
# For 10 executors × 4 cores = 40 cores → use 80-160 partitions
```

Shuffle Partitions

```
# Controls partitions after shuffle operations (joins, groupBy)
spark.conf.set("spark.sql.shuffle.partitions", "200")  # default
spark.conf.set("spark.sql.shuffle.partitions", "50")   # for small data

# Adaptive Query Execution (Spark 3.0+) — auto-tunes partitions
spark.conf.set("spark.sql.adaptive.enabled", "true")
spark.conf.set("spark.sql.adaptive.coalescePartitions.enabled", "true")
```

Partition by Column (for Writing)

```
# Creates directory structure: output/year=2024/month=01/...
df.write.partitionBy("year", "month").parquet("output/")

# Read only specific partitions (partition pruning)
df = spark.read.parquet("output/").filter(col("year") == 2024)
```

⚠️ **Note:** Don't confuse *DataFrame partitions* (in-memory data chunks) with *Hive-style file partitions* (directory structure on disk). They are different concepts!

18. Caching & Persistence

By default, Spark recomputes a DataFrame from scratch every time an action is called. **Caching** stores the DataFrame in memory (or disk) after the first computation, so subsequent actions reuse it. This is critical when you use the same DataFrame multiple times — for example, in iterative ML algorithms or when branching your pipeline.

Cache vs Persist

```
df.cache()                    # store in memory (MEMORY_AND_DISK)
df.persist()                  # same as cache()

from pyspark import StorageLevel
df.persist(StorageLevel.MEMORY_ONLY)          # memory only (fast, may OOM)
df.persist(StorageLevel.MEMORY_AND_DISK)      # spill to disk if needed
df.persist(StorageLevel.DISK_ONLY)            # disk only (slow but safe)
df.persist(StorageLevel.MEMORY_ONLY_SER)      # serialized (less memory, slower)
df.persist(StorageLevel.OFF_HEAP)             # off-heap memory

df.unpersist()                # free cached data when done
```

When to Cache

```
# Good: reused multiple times
df_clean = df.filter(...).dropna().cache()
count1 = df_clean.count()
count2 = df_clean.groupBy("dept").count()

# Check if cached
print(df_clean.is_cached)

# View cached DataFrames in Spark UI
spark.catalog.clearCache()    # clear all caches
```

💡 **Tip:** Cache after expensive operations (joins, aggregations) that are reused. Don't cache DataFrames used only once — it wastes memory.

19. UDFs — User Defined Functions

> **UDFs** let you apply custom Python functions to DataFrame columns. However, they are slow because Spark must serialize data, send it to Python, execute the function, and deserialize results — breaking out of the JVM. Always prefer built-in Spark functions. If you must use a UDF, use **Pandas UDFs** (vectorized UDFs) which are 10-100x faster than regular UDFs.

Regular UDF (Slow — avoid if possible)

```
from pyspark.sql.functions import udf
from pyspark.sql.types import StringType

def categorize(age):
    if age is None: return "unknown"
    if age < 18: return "minor"
    if age < 65: return "adult"
    return "senior"

categorize_udf = udf(categorize, StringType())
df = df.withColumn("category", categorize_udf(col("age")))

# Decorator syntax
@udf(returnType=StringType())
def clean_name(name):
    return name.strip().title() if name else None

df = df.withColumn("clean_name", clean_name(col("name")))
```

Pandas UDF (Fast — vectorized)

```
from pyspark.sql.functions import pandas_udf
import pandas as pd

# Series to Series UDF
@pandas_udf(DoubleType())
def multiply_by_pi(series: pd.Series) -> pd.Series:
    return series * 3.14159

df = df.withColumn("circumference", multiply_by_pi(col("radius")))

# Series to Scalar UDF (for GroupBy)
@pandas_udf(DoubleType())
def weighted_avg(values: pd.Series, weights: pd.Series) -> float:
    return (values * weights).sum() / weights.sum()

df.groupBy("dept").agg(weighted_avg(col("salary"), col("weight")))
```

20. Spark SQL

> **Spark SQL** lets you query DataFrames using standard SQL syntax. You register a DataFrame as a temporary view, then run SQL queries against it. This is great for analysts familiar with SQL and for complex queries that are easier to express in SQL than in the DataFrame API. Spark SQL and the DataFrame API are interchangeable — they produce the same execution plan.

```
df.createOrReplaceTempView("employees")       # session-scoped temp view
df.createOrReplaceGlobalTempView("employees") # cluster-scoped (global_temp.employees)

result = spark.sql("""
    SELECT
        department,
        COUNT(*) AS headcount,
        AVG(salary) AS avg_salary,
        MAX(salary) AS max_salary
    FROM employees
    WHERE is_active = true
    GROUP BY department
    HAVING COUNT(*) > 5
    ORDER BY avg_salary DESC
""")

# Subqueries, CTEs, window functions all work
result = spark.sql("""
    WITH ranked AS (
        SELECT *, RANK() OVER (PARTITION BY dept ORDER BY salary DESC) AS rnk
        FROM employees
    )
    SELECT * FROM ranked WHERE rnk <= 3
""")

spark.catalog.listTables()    # list all registered views
spark.catalog.dropTempView("employees")
```

21. RDD Operations

> **RDD (Resilient Distributed Dataset)** is Spark's original low-level API. DataFrames are built on top of RDDs. RDDs give you full control but lack the optimizations of the DataFrame API (Catalyst optimizer, Tungsten execution engine). Use RDDs only when you need low-level control or are working with unstructured data.

Create RDD

```
rdd = sc.parallelize([1, 2, 3, 4, 5], numSlices=4)  # from Python list
rdd = sc.textFile("path/to/file.txt")               # from text file
rdd = df.rdd                                          # from DataFrame
```

Transformations (Lazy)

```
rdd.map(lambda x: x * 2)              # apply function to each element
rdd.flatMap(lambda x: x.split(" "))   # map then flatten
rdd.filter(lambda x: x > 3)           # keep matching elements
rdd.distinct()                          # remove duplicates
rdd.union(rdd2)                         # combine two RDDs
rdd.intersection(rdd2)                  # common elements
rdd.subtract(rdd2)                      # elements in rdd but not rdd2
rdd.sample(False, 0.1)                  # random 10% sample
rdd.sortBy(lambda x: x, ascending=False)  # sort
rdd.mapPartitions(func)                 # apply function to each partition
rdd.glom()                              # collect each partition into a list
```

Key-Value RDD Operations

```
pairs = rdd.map(lambda x: (x % 3, x))  # create (key, value) pairs
pairs.groupByKey()                       # group values by key
pairs.reduceByKey(lambda a, b: a + b)   # reduce by key (more efficient)
pairs.aggregateByKey(0, lambda a,b: a+b, lambda a,b: a+b)
pairs.sortByKey()                        # sort by key
pairs.countByKey()                       # count per key (returns dict)
pairs.join(pairs2)                       # join two pair RDDs
pairs.leftOuterJoin(pairs2)
pairs.mapValues(lambda v: v * 2)         # transform values only
```

Actions (Trigger Execution)

```
rdd.collect()                  # return all elements as list
rdd.count()                    # count elements
rdd.first()                   # first element
rdd.take(5)                    # first 5 elements
rdd.top(5)                     # top 5 elements
rdd.reduce(lambda a, b: a + b) # reduce to single value
rdd.fold(0, lambda a, b: a + b)# reduce with initial value
rdd.aggregate(...)             # complex aggregation
rdd.foreach(print)             # apply function (no return)
rdd.saveAsTextFile("output/")  # save to text files
```

22. Broadcast Variables & Accumulators

Broadcast variables efficiently share a read-only value (like a lookup dictionary) with all executors. Without broadcasting, Spark sends the variable with every task — very inefficient. With broadcasting, it's sent once per executor and cached.

Accumulators are write-only variables that executors can add to, and only the driver can read. They're used for counters and sums — like counting bad records during a transformation.

Broadcast Variables

```
# Create a broadcast variable
lookup = {"US": "United States", "UK": "United Kingdom", "IN": "India"}
bc_lookup = sc.broadcast(lookup)

# Use in a UDF or map
@udf(StringType())
def expand_country(code):
    return bc_lookup.value.get(code, "Unknown")

df = df.withColumn("country_full", expand_country(col("country_code")))

bc_lookup.unpersist()   # free memory when done
```

Accumulators

```
null_count = sc.accumulator(0)

def count_nulls(row):
    global null_count
    if row["name"] is None:
        null_count += 1

df.rdd.foreach(count_nulls)
print(f"Null names: {null_count.value}")

# Custom accumulator
from pyspark import AccumulatorParam
class ListAccumulator(AccumulatorParam):
    def zero(self, v): return []
    def addInPlace(self, v1, v2): return v1 + v2
```

23. Performance Tuning

Performance in Spark comes down to: minimizing data movement (shuffles), maximizing parallelism, and efficient memory use. The key tools are: the Spark UI (to identify bottlenecks), the query plan (to understand what Spark is doing), and configuration tuning.

Explain Query Plan

```
df.explain()                    # physical plan
df.explain(True)                # logical + physical plan
df.explain("formatted")         # Spark 3.0+ formatted output
df.explain("cost")              # with cost estimates
```

Key Configuration

```
spark.conf.set("spark.sql.shuffle.partitions", "200")       # shuffle partitions
spark.conf.set("spark.sql.adaptive.enabled", "true")          # AQE (Spark 3.0+)
spark.conf.set("spark.sql.adaptive.skewJoin.enabled", "true") # handle skew
spark.conf.set("spark.sql.autoBroadcastJoinThreshold", "10m") # broadcast threshold
spark.conf.set("spark.executor.memory", "8g")
spark.conf.set("spark.executor.cores", "4")
spark.conf.set("spark.default.parallelism", "200")
```

Avoid Common Pitfalls

Anti-Pattern	Better Approach
Using Python UDFs	Use built-in Spark functions or Pandas UDFs
collect() on large data	Write to storage, use show() for sampling
Too many small files	Coalesce before writing
Cartesian joins	Always specify join condition
Not caching reused DFs	Cache after expensive operations
Default shuffle partitions (200) for small data	Reduce to 10-50 for small datasets
Reading entire dataset when filtering	Use partition pruning (partitionBy on write)
Wide schema with many columns	Select only needed columns early

Data Skew Handling

```
# Detect skew: check partition sizes
df.rdd.mapPartitions(lambda it: [sum(1 for _ in it)]).collect()

# Fix skew with salting (add random prefix to key)
import random
df = df.withColumn("salted_key",
    F.concat(col("skewed_key"), F.lit("_"), (F.rand() * 10).cast("int").cast("string"))
)
```

24. Structured Streaming

> **Structured Streaming** is Spark's stream processing engine built on the DataFrame API. It treats a live data stream as an unbounded table that keeps growing. You write the same DataFrame transformations, and Spark continuously processes new data as it arrives. Sources include Kafka, files, sockets, and more.

Read Stream

```
# Read from Kafka
stream_df = spark.readStream \
    .format("kafka") \
    .option("kafka.bootstrap.servers", "host:9092") \
    .option("subscribe", "my-topic") \
    .option("startingOffsets", "latest") \
    .load()

# Read from file directory (new files trigger processing)
stream_df = spark.readStream \
    .format("csv") \
    .schema(schema) \
    .option("path", "input/directory/") \
    .load()
```

Transform Stream

```
from pyspark.sql.functions import window

# Parse Kafka value (bytes -> string -> JSON)
parsed = stream_df.select(
    F.from_json(col("value").cast("string"), schema).alias("data")
).select("dat
```